Document Number: P2071R0
Date: 2020-01-13
Audience: SG16, EWG
Reply-to: Tom Honermann <tom@honermann.net>
Peter Bindels <peterbindels@gmail.com>

Named universal character escapes

Introduction

This proposal continues the effort R. Martinho Fernandes initiated that culminated in P1097R2[P1097R2]. This proposal does not deviate from the general design intent in Fernandes' work, but does deviate in the following specific details:

- This proposal uses UAX44-LM2 for matching names rather than just case-insensitive matching. This is primarily motivated by implementation concerns; ignoring spaces allows for a more efficient implementation.
- This proposal includes a feature test macro.

C++ programmers have been able to portably use characters outside of the basic source character set in character and string literals since the introduction of *universal-character-name*s in C++11. For example:

```
U'\u0100'        // UTF-32 character literal with U+0100 {LATIN CAPITAL LETTER A WITH MACRON}
u8"\u0100\u0300" // UTF-8 string literal with U+0100 {LATIN CAPITAL LETTER A WITH MACRON} U+0300 {COMBINING GRAVE ACCENT}
```

This proposal enables the above literals to be written using Unicode assigned names instead of Unicode code point values.

```
U'\N{LATIN CAPITAL LETTER A WITH MACRON}'                          // Equivalent to U'\u0100'
u8"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT" // Equivalent to u8"\u0100\u0300"
```

Prior presentations of P1097 to EWG-I and EWG received strong encouragement:

- Poll of P1097R1[P1097R1] in EWG-I in San Diego, 2018:
  Do we want named escape sequences?

SF	F	N	A	SA
5	9	7	0	0

- Poll of P1097R2[P1097R2] in EWG in Belfast, 2019:
  EWG wants to encourage further work in this area

SF	F	N	A	SA
8	16	8	1	1

Two areas of concern were raised during discussion in EWG in Belfast, 2019:

- **Implementation impact**
  The Unicode name database (names and aliases), in text form, is ~1.5 MiB and a naive implementation could significantly impact the size of compiler distributions. This was of particular concern to organizations that distribute compilers as part of a distributed build process.
- **Design concerns**
  One EWG member strongly preferred a library based design that would have a smaller impact on the core language. For example, a string interpolation based design.

This paper discusses and links to work completed by Corentin Jabot that investigates implementation impact, though an implementation has not yet been completed. This paper also includes discussion regarding alternative design possibilities.

Motivation

The introduction of *universal-character-name*s in C++11 benefitted programmers by allowing them to portably encode characters outside of the basic source character set without having to resort to use of octal or hexadecimal *escape-sequence*s to explicitly encode code units. However, Unicode code points by themselves do not clearly communicate to readers of the code which character is to be encoded; hence the code comments included with the code examples in the introduction. Allowing programmers to directly use Unicode assigned character names avoids the need for side channel communications, like code comments, that might get out of sync over time.

Use of UTF-8 as the encoding for source files has increased over time, but impediments to adoption remain. For example, Microsoft Visual C++ still defaults to a locale dependent encoding and that encourages limiting source files to ASCII. If the C++ community were to migrate en masse to UTF-8, then one might question whether *universal-character-name*s would become a legacy backward compatibility feature since programmers could reliably type the intended character in their source code directly. And if *universal-character-name*s were to become an anachronism, then what use would be served by introducing a named character escape?

Unicode defines a number of characters that, even when they can be typed directly, can result in confusion. These include invisible characters such as U+200B {ZERO WIDTH SPACE}, combining characters such as U+0300 {COMBINING GRAVE ACCENT}, visually indistinct characters such as U+003B {SEMICOLON} and U+037E {GREEK QUESTION MARK}, and characters with RTL (right-to-left) directionality. Consider how the following string literals containing these characters are rendered. In cases like these, use of escape sequences improves clarity; thus motivation for use of Unicode escape sequences will remain.

```
""  // U+0000200B {ZERO WIDTH SPACE}
""  // U+0000200F {RIGHT-TO-LEFT MARK}
"̀"  // U+00000300 {COMBINING GRAVE ACCENT}
";" // U+0000003B {SEMICOLON}
";" // U+0000037E {GREEK QUESTION MARK}
"´" // U+000000B4 {ACUTE ACCENT}
"́"  // U+00000301 {COMBINING ACUTE ACCENT}
"΄" // U+00001FFD {GREEK OXIA}
"Ω" // U+000003A9 {GREEK CAPITAL LETTER OMEGA}
"Ω" // U+00002126 {OHM SIGN}
"A" // U+00000041 {LATIN CAPITAL LETTER A}
"Α" // U+00000391 {GREEK CAPITAL LETTER ALPHA}
"А" // U+00000410 {CYRILLIC CAPITAL LETTER A}
"Ꭺ" // U+000013AA {CHEROKEE LETTER GO}
"𑊠" // U+0000A4EE {LISU LETTER A}
"𐊠" // U+000102A0 {CARIAN LETTER A}
"𖽀" // U+00016F40 {MIAO LETTER ZZYA}
```

Named character escapes are supported in various forms in other programming languages. The following is the result of a brief survey of various languages. For languages that include such support, more details can be found in the Design considerations section.

Language	**Named character escape support**
C#	No
D	Yes; HTML 5 named character references
Go	No
Java	No
Javascript	No
Perl	Yes; Unicode names, aliases, and named sequences
PHP	No
Python	Yes; Unicode names and aliases
Raku	Yes; Unicode names, aliases, named sequences, and emoji sequences
Ruby	No
Rust	No
Swift	No
Visual Basic	No

Design considerations

There are numerous choices for how support for named characters can be integrated into C++. Useful questions for making design choices include:

- Which names will be recognized? Can multiple names for the same character exist?
- How will names be matched? Must they be exact? Case insensitive?
- How will support for new names affect backward compatibility?

- How will the requirement for a name database impact implementations?
- What syntax to use?
- What is existing practice in other languages?

This section analyzes the various options considered for this proposal.

Syntax

Named character escapes are proposed as a more readable alternative to universal-character-names. As such, it is desirable that they be similar in syntax to universal-character-names and other existing escape sequences.

The syntax proposed by Fernandes in P1097R2[P1097R2] is modeled after the syntax adopted for Python and consists of a `\N` escape introducer followed by a name enclosed in curly brackets. For example:

```
'\N{LATIN CAPITAL LETTER A}'
"\N{LATIN CAPITAL LETTER A WITH MACRON}"
```

Other choices for the escape introducer are possible; the Backward compatibility section discusses some possible motivation for preferring `\u` and/or `\U` and the Proposal options section includes this alternate syntax as an option.

Options for recognized names and how to match them are discussed in subsequent sections.

As proposed, only one name is allowed per named character escape, but that is an artificial limitation. Raku allows a sequence of comma separated names to be specified in a single escape. This is a natural extension if names are permitted to identify sequences of characters instead of a single character. The following would all be equivalent. This proposal leaves this option to a future extension; see the Possible future extensions section.

```
"\N{LATIN CAPITAL LETTER A WITH MACRON, COMBINING GRAVE ACCENT}"
"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT}"
"\u0100\u0300"
```

Perl and Raku both allow Unicode code point numbers to be specified as character names and could enable a syntax that avoids the strict 4 or 8 number requirements of universal-character-names as well as the natural `U+NNNN` style frequently used to identify Unicode characters. The following could all be equivalent. This proposal also leaves this option for a future extension as discussed in the Possible future extensions section.

```
"\N{U+0100}"
"\N{U+100}"
"\N{U+00000100}"
"\N{0x0100}"
"\N{256}"
"\u0100"
```

Name sources

A named character escape feature is not particularly useful unless accompanied by at least one source of character names. The following list contains sources of character names that are consulted by at least one implementation of named character escapes in another programming language.

- Unicode assigned names (synchronized with ISO/IEC 10646)
  https://www.unicode.org/Public/12.0.0/ucd/NamesList.txt
- Unicode aliases (synchronized with ISO/IEC 10646)
  https://www.unicode.org/Public/12.0.0/ucd/NameAliases.txt
- Unicode named sequences (synchronized with ISO/IEC 10646)
  https://www.unicode.org/Public/12.0.0/ucd/NamedSequences.txt
- Emoji ZWJ sequences
  https://www.unicode.org/Public/emoji/4.0/emoji-zwj-sequences.txt
- Emoji sequences
  https://www.unicode.org/Public/emoji/4.0/emoji-sequences.txt
- HTML named character references
  https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references

The first three are defined by the Unicode Consortium, part of the Unicode standard, and synchronized with ISO/IEC 10646. The names specified in each are designed in concert, share a common namespace, are immutable once published, and Unicode guarantees no conflicts between them. See the Unicode character encoding stability policy[UCESP] for more details. These sources are consulted for named character escapes in Perl, Python, and Raku.

The next two sources specify emoji character sequences. Though produced by the Unicode Consortium, they are not part of the Unicode standard, and are not covered by the Unicode character encoding stability policy[UCESP]. These two sources don't technically provide names; they provide optional descriptions. The provided descriptions use characters, particularly `:` and `,`, that are disallowed in the names provided by the first three sources. These sources are consulted for named character escapes in Raku.

The last source is the specification of names recognized for use as named character references in HTML documents. This source is used for the implementation of named character escapes in the D programming language.

The stability guarantees offered by the Unicode standard are a strong motivator for their use and, as such, this proposal adopts them as the name sources to use.

The list of Unicode assigned names associates at most one name with each character. There are some characters that are not assigned a name in this list, for example, U+0080 is simply listed as a `<control>` character with no name. In some of these cases, the Unicode aliases list provides one or more names. For example, U+0080 has assigned aliases of `PADDING CHARACTER` (a figment alias) and `PAD` (an abbreviation alias).

Unicode aliases provide another critical service. As mentioned above, once assigned, names are immutable. Corrections are only offered by providing an alias. Aliases come in five varieties:

- **correction**
  Aliases for cases where an incorrect assigned name was published. For example, U+FE18 has an assigned name of `PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRAKCET` and a correction alias of `PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRACKET` (note the typo correction).
- **control**
  Aliases for various control characters. For example, U+0000 as a control alias of `NULL`.
- **alternate**
  Aliases for widely used alternate names. For example, `BYTE ORDER MARK` for U+FEFF.
- **figment**
  Aliases for names that were documented, but never accepted in a standard. For example, `HIGH OCTET PRESET` for U+0081.
- **abbreviation**
  Aliases for common abbreviations. For example, `NBSP` for U+00A0.

It is conceivable that implementors could desire, or be requested to, support additional implementation-defined names; perhaps including from the additional sources listed above. Since new characters and names will continue to be added to the Unicode standard, caution is warranted to avoid the possibility of introducing conflicting names over time. The description of the UAX44-LM2 name matching algorithm describes a historical case of how such a conflict once occurred. Any support for additional names should ensure that they occupy a non-overlapping namespace with the Unicode assigned names. Out of caution, this proposal disallows additional implementation-defined names.

Name matching

Names can be finicky things. Having to remember whether a name is, for example, `ZERO WIDTH SPACE` or `ZERO-WIDTH SPACE` is likely to frustrate programmers. Some programmers might prefer `zero width space`.

Unicode provides a straight forward algorithm for matching names with various allowances including case-insensitivity, omission of some hyphens (`-`), and substitution of underscore (_) for space characters. UAX44-LM2 is included in the Unicode standard via Unicode Standard Annex #44[UAX#44].

The UAX44-LM2 matching rule would accept any of the following names as a match for U+200B {ZERO WIDTH SPACE}

```
ZERO WIDTH SPACE
ZERO-WIDTH SPACE
zero-width space
ZERO width S P_A_C E
```

Portable names

Portably using named character escapes will require implementations to agree on a minimum version of the name sources.

Thanks to the adoption of P1025R1[P1025R1] in Rapperswil, 2019, the C++ standard has a normative floating reference to ISO/IEC 10646[ISO/IEC10646], the ISO/IEC standard that specifies a subset of what is specified in the Unicode standard and is kept synchronized with it. ISO/IEC 10646:2017 includes the Unicode assigned names (in section 33), name aliases (in section 33), and named character sequences (in section 27).

The floating reference to ISO/IEC 10646 indicates a dependence on the version that is current at the time of standardization. Thus, conformance with the C++ standard will require conformance with the latest available publication of ISO/IEC 10646.

Implementors must be allowed, and encouraged, to conform to more recent versions of ISO/IEC 10646 as they are published.

Existing practice

Support for named escape sequences exists in several programming languages. The following details of existing practice were obtained from these documentation sources. The author has not verified the accuracy of this information.

Language	Documentation link
D	https://dlang.org/spec/lex.html#StringLiteral
Perl	https://perldoc.perl.org/charnames.html
Python	https://docs.python.org/3.8/reference/lexical_analysis.html#literals

Raku	https://docs.raku.org/language/unicode#Entering_unicode_codepoints_and_codepoint_sequences

Capabilities vary across languages:

Language	Name sources	Comma separated names	Name matching	Matches code point numbers
D	HTML 5	No	Exact match?	No
Perl	Unicode names Unicode name aliases Unicode named sequences registered custom aliases	No	Optionally, script qualified short names Optionally, loose matching (case insensitive, ignore underscore, most spaces, and most non-medial hyphens)	Yes
Python	Unicode names Unicode name aliases	No	Case-insensitive	No
Raku	Unicode names Unicode name aliases Unicode named sequences emoji ZWJ sequences emoji sequences	Yes	Exact match?	Yes

Examples:

Language	Code
D	`"\&Amacr;"`
Perl	`"\N{LATIN CAPITAL LETTER A WITH MACRON}"` `"\N{U+0100}"`
Python	`"\N{LATIN CAPITAL LETTER A WITH MACRON}"`

Raku	`"\c[LATIN CAPITAL LETTER A WITH MACRON]"`
	`"\c[256]"`
	`"\c[LATIN CAPITAL LETTER A WITH MACRON,COMBINING GRAVE ACCENT]"`
	`"\c[LATIN CAPITAL LETTER A WITH MACRON AND GRAVE]"`

Backward compatibility

Escape sequences beyond those required in the standard are conditionally-supported ([[lex.ccon]p7](#)). For implementations that currently define a meaning for `\N` in character or string literals, the use of `\N` in this proposal is technically a breaking change.

Gcc, Clang, and Microsoft Visual C++ all accept `\N` as an escape sequence with the semantic effect of substituting `N` such that `"\N{xxx}"` is equivalent to `"N{xxx}"`. However, they each emit a warning regarding an unrecognized escape sequence, so reliance on this behavior is not likely to be common. Still, there are likely to be some uses in the wild (probably some percentage of that were intended to be `\n`).

Another option would be to reuse the `\u` and/or `\U` introducer used for *universal-character-name*s. Gcc and Clang both reject code like `"\u{xxx}"` and `"\U{xxx}"` as containing ill-formed *universal-character-name*s. However, Microsoft Visual C++ accepts such uses without a warning and treats them as equivalent to `"u{xxx}` and `"U{xxx}"` respectively.

The implementation divergence that occurs for the `\u` and `\U` cases above suggests that repurposing them may result in less backward compatibility. Use of `\u` and/or `\U` would potentially require more wording changes to distinguish named character escapes from *universal-character-name*s, but would be unlikely to pose a significant additional impact to implementors.

For now, this proposal adheres to Fernandes' original design and retains use of `\N` as the introducer for named character escapes.

Implementor impact

The sources of character names listed in the Name sources section do not constitute big data by today's standards, but that does not mean that the volume of data and potential for impact to compiler distributions and compiler performance is insignificant. As mentioned earlier, some

organizations have valid technical reasons to be sensitive to the size of the compiler distributions they use; in a distributed build environment that distributes compilers, the size of the distribution impacts latency and can therefore negatively impact build times.

The combined size of the Unicode 12.0 text files containing the Unicode assigned names, aliases, and named character sequences is approximately 1.5 MiB. A naive implementation might contribute 2+ MiB of code/data to a compiler. Some EWG members indicated that amount of increase is a cause for concern.

Fortunately, naive implementations are not the only option. Corentin Jabot has done some excellent work to demonstrate that an implementation should be possible that increases the code/data size of a compiler by less than 300 KiB. See the Implementation experience section for details. Corentin's approach is promising, but the additional complexity caries additional implementation cost and maintenance.

Staying up to date with new Unicode releases will also, of course, pose an additional cost on implementors.

Design alternatives

As indicated previously, at least one EWG member in Belfast was strongly interested in a more general core language feature, presumably a string interpolation facility, that would allow named character escapes to be implemented as a library feature. Such a feature could take many forms, but might look something like the following where `\{` is an escape sequence followed by a call to a `constexpr` function named `nce` with arguments passed in some form.

```
"\{nce(LATIN CAPITAL LETTER A WITH GRAVE)}"
```

Such a feature could certainly be implemented, but would seem to necessarily be more verbose and would necessitate inclusion of appropriate headers; headers that would be quite large in the case of a named character database or that would make use of a compiler intrinsic; which would put the complexity back in the compiler (though in implementation-defined territory rather than in standard core language). The verbosity concern could potentially be reduced by introducing core language sugar for lowering the proposed syntax to the example string interpolation syntax above.

Proposal

The wording included in this proposal is for the following design:

- Context:
  - Named character escapes are valid only in character and string literals.
- Syntax:
  - `\N{xxx}` where the name is substituted for `xxx`.
- Name sources:
  - ISO/IEC 10646 assigned names.
  - ISO/IEC 10646 assigned name aliases.
  - No allowance for additional implementation-defined names.
- Name matching:
  - As specified by rule UAX44-LM2 in UAX#44[UAX#44].
- Feature test macro:
  - `__cpp_named_character_escapes`

Proposal options

The following options are *not* currently proposed, but could be adopted as modifications of the current proposal.

1. Instead of `\N`, reuse the `\u` and/or `\U` introducers from *universal-character-name*s to introduce a named character escape. For example:
   - `"\u{LATIN CAPITAL LETTER A WITH GRAVE}"`
   - `"\U{LATIN CAPITAL LETTER A WITH GRAVE}"`

   See the Backward compatibility section for more discussion of this option.
2. Allow names to match ISO/IEC 10646 named sequences such that the following would be equivalent:
   - `"\N{LATIN CAPITAL LETTER A WITH MACRON AND GRAVE}"`
   - `"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT"`
   - `"\u0100\u0300"`

Possible future extensions

The following options are *not* currently proposed but could be considered for future extension.

1. Allow named character escapes to be used outside of character and string literals (e.g., in identifiers) analogously to *universal-character-name*s.
2. Allow comma separated names. For example:
   - `"\N{LATIN CAPITAL LETTER A WITH MACRON, COMBINING GRAVE ACCENT}" // Equivalent to "\u0100\u0300"`
3. Allow code point numbers as names. For example:

    - `"\N{U+00C0}" // U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}`
    - `"\N{0x00C0}" // U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}`
    - `"\N{192}" // U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}`
4. Allow names to match Unicode emoji named sequences
5. Allow names to match Unicode emoji ZWJ named sequences
6. Allow names to match HTML 5 named character references by surrounding them with `&` and `;`. For example:
    - `"\N{&Agrave;}" // U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}`

Implementation experience

This proposal has not yet been implemented in an existing compiler. However, the implementation concerns raised in Belfast prompted Corentin Jabot to conduct an experiement to determine how small the implementation overhead, in terms of data and code within the compiler, could be reduced to. His blog post[CJ_BLOG] on the experiment reported that he was able to implement a function (`cp_from_name`) that accepts a Unicode 12.0 name or name alias and returns a code point value in under 300 KiB. His implementation is available in the `cp_to_name` branch of his `ext-unicode-db` GitHub repository at https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp[CJ_IMPL].

Acknowledgements

Thank you to R. Martinho Fernandes for taking the initiative to research and first propose support for named character escapes and for contributing his considerable expertise in general to SG16.

Thank you to Corentin Jabot for the excellent work he did experimenting with and analyzing implementation impact. Without his work, the data necessary to respond to the implementation concerns raised in Belfast would not have been available at this time, thereby delaying further progress on this proposal.

Thank you to Peter Bindels and Corentin Jabot for providing feedback on an initial draft that I delivered to then less than two hours before the Prague pre-meeting mailing deadline!

References

[CJ_BLOG]	Corentin Jabot, "Storing Unicode: Character Name to Codepoint Mapping", 2019. https://cor3ntin.github.io/posts/cp_to_name
[CJ_IMPL]	Corentin Jabot, "ext-unicode-db", 2019. https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp
[ISO/IEC10646]	"Information technology — Universal Coded Character Set (UCS)", ISO/IEC 10646:2017, 2017. https://www.iso.org/standard/69119.html
[N4835]	"Working Draft, Standard for Programming Language C++", N4835, 2019. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4835.pdf
[P1025R1]	Steve Downey, et al. "Update The Reference To The Unicode Standard", P1025R1, 2018. https://wg21.link/p1025r1
[P1097R1]	R. Martinho Fernandes, "Named character escapes", P1097R1, 2018. https://wg21.link/p1097r1
[P1097R2]	R. Martinho Fernandes, "Named character escapes", P1097R2, 2019. https://wg21.link/p1097r2
[P2029R0]	Tom Honermann, "Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals", P2029R0, 2020. https://wg21.link/p2029r0
[UCESP]	"Unicode Character Encoding Stability Policies", 2017. https://www.unicode.org/policies/stability_policy.html
[UAX#44]	Ken Whistler and Laurențiu Iancu, "Unicode Standard Annex #44 - Unicode Character Database", Revision 24, Unicode 12.0.0, 2019. https://www.unicode.org/reports/tr44/tr44-24.html

Core wording

These changes are relative to N4835[N4835].

If P2029R0[P2029R0] were to be adopted, substantial wording updates will be required.

☐ Hide inserted text
☐ Hide deleted text

Change in 5.2 [lex.phases] paragraph 5:

> Each basic source character set member in a character literal or a string literal, as well as each escape sequence ~~and~~, *universal-character-name*, and *named-escape-sequence* in a character literal or a non-raw string literal, is converted to the corresponding member of the execution character set ([lex.ccon], [lex.string]); if there is no corresponding member, it is converted to an implementation defined member other than the null (wide) character. [8]

Change in 5.13.3 [lex.ccon]:

> character-literal:
> encoding-prefix$_{opt}$ ' c-char-sequence '
>
> encoding-prefix: one of
> `u8 u U L`
>
> c-char-sequence:
> c-char
> c-char-sequence c-char
>
> c-char:
> any member of the basic source character set except the single-quote `'`, backslash `\`, or new-line character
> escape-sequence
> universal-character-name
>
> escape-sequence:
> simple-escape-sequence
> octal-escape-sequence
> hexadecimal-escape-sequence
> named-escape-sequence
>
> simple-escape-sequence: one of
> `\' \" \? \\`
> `\a \b \f \n \r \t \v`
>
> octal-escape-sequence:
> `\` octal-digit
> `\` octal-digit octal-digit
> `\` octal-digit octal-digit octal-digit
>
> hexadecimal-escape-sequence:
> `\x` hexadecimal-digit
> hexadecimal-escape-sequence hexadecimal-digit
>
> named-escape-sequence:
> `\N{` n-char-sequence `}`
>
> n-char-sequence:
> n-char
> n-char n-char-sequence
>
> n-char: one of
> `A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
> `a b c d e f g h i j k l m n o p q r s t u v w x y z`
> `0 1 2 3 4 5 6 7 8 9`
> `_` - space

Change in 5.13.3 [lex.ccon] paragraph 7:

> Certain non-graphic characters, the single quote `'`, the double quote `"`, the question mark `?`,[19] and the backslash `\`, can be represented according to Table 8. The double quote `"` and the question mark `?`, can be represented as themselves or by the escape sequences `\"` and `\?` respectively, but the single quote `'` and the backslash `\` shall be represented by the escape sequences `\'` and `\\` respectively. Escape sequences in which the character following the backslash is not listed in Table 8 are conditionally-supported, with implementation-defined semantics. An escape sequence specifies a single character.
>
> Table 8: Escape sequences [tab:lex.ccon.esc]
>
new-line	NL(LF)	`\n`
horizontal tab	HT	`\t`
vertical tab	VT	`\v`

backspace	BS	`\b`
carriage return	CR	`\r`
form feed	FF	`\f`
alert	BEL	`\a`
backslash	\	`\\`
question mark	?	`\?`
single quote	'	`\'`
double quote	"	`\"`
octal number	ooo	`\ooo`
hex number	hhh	`\xhhh`
named escape sequence	named character	`\N{xxx}`

Add a new paragraph (X) after 5.13.3 [lex.ccon] paragraph 9:
Drafting Note: Associated character names and character name aliases are listed in section 33 of ISO/IEC 10646:2017. Named UCS sequence identifiers are listed in section 27.

A *named-escape-sequence* is translated to the encoding, in the appropriate execution character set, of the character or character sequence associated with the ISO/IEC 10646 `associated character name` or `character name alias` that matches the name specified by the *n-char-sequence*. Matching of names is performed by:

(X.1) — removing all medial hyphens.

(X.2) — removing all space and underscore characters.

(X.3) — lowercasing all capital letters.

If no name is matched, then the program is ill-formed. If the matched name is `HANGUL JUNGSEONG OE`, then steps 2 and 3 are performed against the name `HANGUL JUNGSEONG O-E` and, if the names match, U+1180 {HANGUL JUNGSEONG O-E} is encoded, otherwise U+116C {HANGUL JUNGSEONG OE} is encoded. Otherwise, the character associated with the matched name is encoded. [*Note:* The special handling of U+1180 {HANGUL JUNGSEONG O-E} resolves an ambiguity in the matching algorithm; this is the only case of ambiguity. — *end note*]

Change in 5.13.5 [lex.string] paragraph 14:

Escape sequences and *universal-character-name*s in non-raw string literals have the same meaning as in character literals ([lex.ccon]), except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a *universal-character-name* or *named-escape-sequence* in a UTF-16 string literal may yield a surrogate pair. In a narrow string literal, a *universal-character-name* or *named-escape-sequence* may map to more than one `char` or `char8_t` element due to *multibyte encoding*. The size of a `char32_t` or wide string literal is the total number of escape sequences, *universal-character-name*s, *named-escape-sequences*, and other characters, plus one for the terminating `U'\0'` or `L'\0'`. The size of a UTF-16 string literal is the total number of escape sequences, *universal-character-name*s, *named-escape-sequences*, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] Within `char32_t` and `char16_t` string literals, any *universal-character-name*s shall be within the range `0x0` to `0x10FFFF`. The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each *universal-character-name*, *named-escape-sequences*, plus one for the terminating `'\0'`.

Change in table 17 of 15.11 [cpp.predefined] paragraph 1.8:
Drafting note: the final value for the `__cpp_named_character_escapes` feature test macro will be selected by the project editor to reflect the date of approval.

Table 17 — Feature-test macros [tab:cpp.predefined.ft]

Macro name	Value
[…]	[…]
__cpp_modules	201907L
__cpp_named_character_escapes	XXXXXXL **** placeholder ****
__cpp_namespace_attributes	201411L
[…]	[…]